

DEX Operator Handbook

For operators already running a Reality testnet validator: how to run a node in the swaps DEX cluster, and how to trade through the Portal desktop app — single swaps and chained multi-swaps across ETH, BTC, NET, and rApp tokens.

01 What a DEX node is

The DEX is not part of the core Reality node — it runs as its own **rApp chain** registered with Reality as a state channel. Swap *requests* and *matching* live on Reality itself; the DEX chain picks up matched swaps, **verifies the foreign leg against the real foreign chain** (an Ethereum RPC, a Bitcoin node, or another rApp's token ledger), accepts the fulfilment, and reports back to Reality, which settles the NET side.

```
Reality L1 → match on Reality L0 → handoff to DEX L0 → foreign leg paid → fulfilment
on DEX L1 → verified vs foreign chain → NET settles on Reality
```

A full DEX validator is a **pair of processes**:

LAYER	ROLE	PORTS (PUBLIC / P2P / CLI)
DEX L0	snapshot consensus, swap handoff state, token registry, <code>/dex</code> + <code>/rapp-ledger</code> APIs	9100 / 9101 / 9102
DEX L1	accepts <code>SwapFulfilmentTransaction</code> and rapp-token transfers	9110 / 9111 / 9112

Every DEX validator independently verifies foreign transactions, so **each node needs its own access to the foreign chains**: an Ethereum JSON-RPC endpoint and a Bitcoin Core RPC. A node that can't see a chain can't validate fulfilments in that currency and falls out of consensus on those blocks.

02 Prerequisites

- A machine meeting your current Reality validator spec, with Docker + docker-compose.
- **Matching artifacts:** the DEX and Reality network must run the coordinated release with rApp-token support (the reality `0.0.0+1140-rapptoken` line). Mixed versions fail with JSON decode errors at the network boundary.
- From the network operators (testnet channel):
 - the DEX **seed node** host and peer IDs (`L0_INITIAL_HOST` , `L0_INITIAL_ID` , `L1_INITIAL_HOST` , `L1_INITIAL_ID`)
 - the testnet **Ethereum RPC URL** and **Bitcoin RPC** (URL, username, password, network)
 - the DEX **rApp address** (`NET...`) — needed by wallets, not by your node
- A **p12 keypair per process** (L0 and L1), generated with the reality keytool you already use:

```
java -jar reality-keytool.jar generate \  
  --keystore my-dex-l0.p12 --alias alias --storepass <pass> --keypass <pass>
```

03 Build or pull the images

```
git clone <rapp-eth-swaps repo> && cd rapp-eth-swaps  
sbt assembly  
docker compose -f docker-compose.rapp-eth-separate.yml build
```

This produces the `rapp-eth-l0` / `rapp-eth-l1` images (dockerfiles under `local-dev/`). If prebuilt images are published for the testnet release, pull those instead — the version-matching rule still applies.

04 Configure a joining validator pair

`docker-compose.rapp-eth-separate.yml` is the local 6-node reference cluster; on testnet you run **one L0 + one L1 validator joining the existing cluster**. The entrypoints read:

VARIABLE	MEANING
NODE_COMMAND	run-validator for joining nodes (run-genesis / run-initial-validator belong to the seed)
PORT_BASIS	port offset — ports are 9100 + PORT_BASIS×10 ; use 0 (L0) and 1 (L1) on a dedicated machine
L0_INITIAL_HOST / L0_INITIAL_ID	the seed DEX L0 (operator-provided)
L1_INITIAL_HOST / L1_INITIAL_ID	the seed DEX L1 (L1 nodes only)
ETHEREUM_URL	Ethereum JSON-RPC your node verifies ETH legs against
BITCOIN_RPC_URL ..._USERNAME ..._PASSWORD BITCOIN_NETWORK	Bitcoin Core RPC for BTC legs (regtest testnet3 mainnet)
CL_KEYSTORE / CL_KEYALIAS / CL_PASSWORD	your p12 (mounted into the container), alias, password

```

services:
  dex-l0:
    image: rapp-eth-l0:<release>
    environment:
      - NODE_COMMAND=run-validator
      - PORT_BASIS=0
      - L0_INITIAL_HOST=<seed-host>
      - L0_INITIAL_ID=<seed-l0-peer-id>
      - ETHEREUM_URL=<eth-rpc-url>
      - BITCOIN_RPC_URL=<btc-rpc-url>
      - BITCOIN_RPC_USERNAME=<btc-user>
      - BITCOIN_RPC_PASSWORD=<btc-pass>
      - BITCOIN_NETWORK=testnet3
      - CL_KEYSTORE=keys/my-dex-l0.p12
      - CL_PASSWORD=<keystore-pass>
    volumes:
      - ./my-dex-l0.p12:/app/keys/my-dex-l0.p12:ro
    ports: ['9100:9100', '9101:9101']

  dex-l1:
    image: rapp-eth-l1:<release>
    environment:
      - NODE_COMMAND=run-validator
      - PORT_BASIS=1
      - L0_INITIAL_HOST=<seed-host>
      - L0_INITIAL_ID=<seed-l0-peer-id>
      - L1_INITIAL_HOST=<seed-host>
      - L1_INITIAL_ID=<seed-l1-peer-id>
      - ETHEREUM_URL=<eth-rpc-url>
      - BITCOIN_RPC_URL=<btc-rpc-url>
      - BITCOIN_RPC_USERNAME=<btc-user>
      - BITCOIN_RPC_PASSWORD=<btc-pass>
      - BITCOIN_NETWORK=testnet3
      - CL_KEYSTORE=keys/my-dex-l1.p12
      - CL_PASSWORD=<keystore-pass>
    volumes:
      - ./my-dex-l1.p12:/app/keys/my-dex-l1.p12:ro
    ports: ['9110:9110', '9111:9111']

```

The entrypoint waits for the seed node, announces itself, and joins automatically.

05 Verify your node

```
# L0 following snapshots (ordinal should advance):
curl -s http://localhost:9100/global-snapshots/latest/ordinal

# swap handoff state ({} when nothing is matched):
curl -s http://localhost:9100/global-snapshots/matched-swaps

# token registry:
curl -s http://localhost:9100/dex/tokens

# L1 answering queries:
curl -s http://localhost:9110/transactions/last-reference/<any-NET-address>
```

Healthy = the ordinal advancing in step with the other DEX nodes and no recurring `Could not decode JSON` or foreign-RPC errors in `docker logs`.

06 Swapping in the Portal UI

Portal (the Reality desktop app) contains the swaps UI. One-time setup: open **Swaps** — the **DEX connection** card asks for the operator-published values: the DEX rApp address (`NET...`) and the DEX L0/L1 endpoints (`host:9100` , `host:9110` , one per line — your own node's address works and keeps your traffic local).

The one rule that surprises everyone: matching is **exact**. Your order matches only a counterparty whose NET amount and foreign amount are *exactly equal* to yours, on the same pair. The Order Book tab exists for this — find an open order and press **Take**, which pre-fills the perfectly mirrored order.

Buying a foreign asset (NET → ETH / BTC / token)

1. Swap tab → direction *Give NET → receive X* → amounts → **your own X address** (the counterparty is forced to pay this address — the DEX verifies it).
2. Review & sign → wallet password.
3. My Swaps: *On book* → *Matched* → *Handed off* → *Fulfilment* → *Settled*. Nothing else to do — the seller pays you and fulfils; settlement moves your NET.

Selling a foreign asset (ETH / BTC → NET)

Same flow with the direction flipped (your foreign address = the account you send from). After *Handed off*, an action card appears:

- **Manual** — send *exactly* the shown amount to the shown buyer address from any wallet, paste the transaction hash, **Submit fulfilment**, password.
- **Automatic** (Swaps → settings → *Automation*) — press **Send automatically**: Portal sends the ETH (natively, using the ETH key stored in your encrypted vault) or the BTC (via Bitcoin wallet RPC), waits for confirmations, and submits the fulfilment itself. Automation needs an ETH RPC URL + a stored ETH key, and/or Bitcoin wallet RPC credentials.

Selling a **rApp token** never needs an external wallet: the card is just *Authorize transfer* — one password, and Portal does the on-ledger transfer and fulfilment.

Multi-swaps (e.g. ETH → NET → BTC)

Multi-Swap tab → pick a route (ETH→NET→BTC , BTC→NET→ETH , token→NET→token) → amounts and addresses for both legs → **one password authorizes the whole chain**. Leg 1 submits immediately; when it settles, leg 2 submits itself. With automation configured the entire chain is hands-off; without it there is one manual send-and-paste step on the selling leg. Keep the app running — after a restart mid-chain, the leg shows *Re-authorize* (one password re-arms it).

Before placing orders:

- **No cancellation exists**. An unmatched order sits on the book until taken; a matched swap locks the buyer's `amount + fee` and the seller's `stake + fee` until fulfilled.
- Amounts are exact-match — coordinate with your counterparty or use **Take**.
- The buyer's foreign address is enforced by verification; the seller's is informational.

07 Troubleshooting

SYMPTOM	LIKELY CAUSE
Could not decode JSON in node logs	artifact version mismatch between DEX, Reality nodes, or wallet — align on the coordinated release
Fulfilment rejected, no match found	submitted before the swap was handed off to the DEX chain — Portal waits automatically; scripts should poll <code>:9100/global-snapshots/matched-swaps</code> first

SYMPTOM	LIKELY CAUSE
ETH/BTC fulfilment rejected	foreign tx unconfirmed, wrong destination (must pay the <i>buyer's</i> address), or wrong amount (destination must receive the exact <code>amountForeign</code>)
Swap stuck at <i>Matched</i>	DEX cluster hasn't ingested the handoff — check DEX L0 health / ordinal progression
Order never matches	no exactly-equal counterparty — check the order book for near-misses
Log timestamps look wrong	container clocks are UTC and can skew from the host — correlate by ordinals/hashes, not wall time